

Sesión 3

Detección de variantes con IA

Reglas YARA y Sigma desde informes CTI, con LLM local y validación — a prueba de variantes

Del informe de amenaza a la regla funcional que caza a toda la familia, no a un solo archivo

Jawy Andrés Romero Pinto · Módulo 92-EAN · Universidad Ean · Educación Continua

2 horas · 1h teoría + 1h laboratorio

Agenda de la sesión

Del informe de inteligencia (CTI) a reglas de detección validadas



De la Sesión 2 a la 3

Del qué está dañado a quién fue y sus variantes.



La pirámide del dolor

Qué le cuesta cambiar al actor: de hash a TTPs.



YARA: firmas estáticas

Anatomía: meta, strings (texto/hex) y condition.



Sigma: comportamiento

El 'YARA de los logs'; una regla, N SIEMs.



IA que escribe reglas

Borrador con LLM local + auditoría con banco de pruebas.



Laboratorio (2ª hora)

Caso Locked3D: YARA, Sigma y copiloto auditado.

La estrella polar (variantes)

La pregunta del curso, cuando el actor recompila



«El ransomware volvió con otra cara: ¿cómo detecto a TODA la familia —presente y futura— sin marcar lo legítimo?»



Cazar familias, no archivos

Un hash detecta un archivo; una regla, la familia.



Indicadores que duelen

Priorizar lo caro de cambiar: formato, mutex, TTPs.



Validar contra lo benigno

Una regla sin banco de pruebas es una promesa.



Copiloto, no piloto

El LLM redacta; el validador y tú deciden.

De la Sesión 2 a la Sesión 3

Del archivo dañado al actor que lo dañó — y sus variantes

Sesión 2 · integridad

- Scoring del backup completo
- Entropía, magic y hashing difuso
- Responde QUÉ está dañado
- Ideal para HALLAR la copia limpia

Sesión 3 · detección

- Del daño al ACTOR: caso Locked3D
- Reglas YARA (archivos) y Sigma (logs)
- Responde QUIÉN fue y dónde más está
- Ideal para CAZAR variantes futuras

El reto: el actor recompila más rápido que tú

Un hash caza UN archivo — tú necesitas cazar la familia

v3.1

recompilada en semanas: strings y dominios nuevos

1

byte cambiado basta para invalidar un hash exacto

2

indicadores sobreviven entre variantes: cabecera y mutex

IA

redacta el borrador de la regla; el analista la audita

La respuesta es apuntar a lo que al actor le DUELE cambiar: la pirámide del dolor.

La pirámide del dolor

Qué le cuesta cambiar al actor — ahí apunta tu regla

Barato de cambiar (frágil)

- Hash del binario: 1 byte y muere.
- Dominios (.onion): rotan por campaña.
- Strings del core: una recompilación.
- Firmas de un solo IOC caducan en semanas.

Caro de cambiar (estable)

- Mutex de infección única: toca su lógica.
- Cabecera del formato de cifrado (L3D!).
- TTPs: borrar shadows, limpiar logs.
- Reglas sobre esto sobreviven a la v3.1.

YARA · anatomía de una regla

Firmas estáticas sobre archivos: texto, hex y lógica

Los tres bloques

- meta: autor, descripción, referencia CTI
- strings: texto (\$mutex) y hex ({ 4C 33 44 21 })
- condition: la lógica que decide el disparo
- Se valida con: `yara -r -s regla.yar directorio/`



La regla del caso Locked3D

- \$magic = { 4C 33 44 21 } — cabecera L3D!
- \$core, \$mutex, \$nota — strings del informe
- condition: \$magic at 0 or 2 of (...)
- Caza v3, v3.1 y la build empacada

Diseñar la condition es diseñar el error

Sensibilidad vs. precisión: tú eliges qué error pagar

'any of them' (floja)

- Dispara con un solo string cualquiera.
- Marca el runbook de TI que solo MENCIONA la nota.
- Falso positivo en producción = SOC a las 3 a.m.
- Muy sensible, nada confiable.

El equilibrio del lab

- La cabecera al offset 0 basta por sí sola.
- Sin cabecera: exige AL MENOS 2 señales (2 of).
- 0 falsos positivos y aún caza la empacada.
- Cada umbral es una decisión de costo.

Sigma · el YARA de los logs

Detectar el comportamiento que el ransomware no puede evitar

Anatomía (.yml)

- logsource: process_creation (Windows)
- detection: selecciones + condition
- Modificadores: contains, endswith, |all
- level: severidad para el SOC

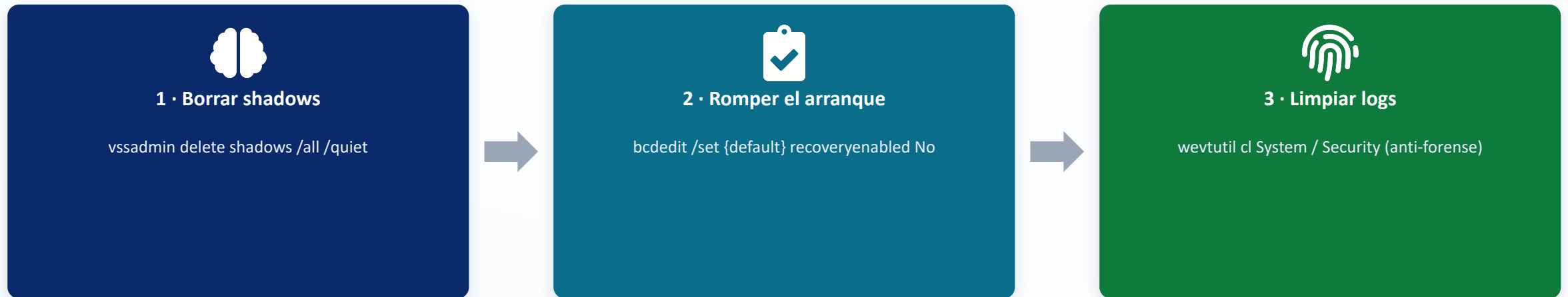


Una regla, N SIEMs

- sigma convert -t splunk regla.yml
- Splunk, Elastic, Sentinel: el mismo YAML
- SigmaHQ: miles de reglas comunitarias
- Adaptar y validar > reinventar

Lo que el ransomware TIENE que hacer

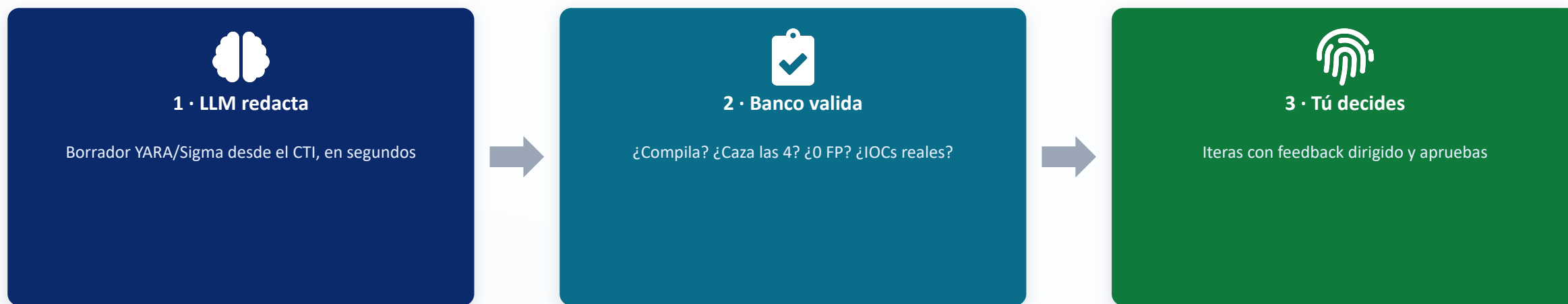
Inhibir tu recuperación (T1490) — justo ahí lo cazas



Cuidado con el contexto: el agente de backup usa `vssadmin list/create` a diario. La regla exige `delete + shadows` en el MISMO comando, o filtra la cuenta de backup. Distinguir, no prohibir.

El copiloto escribe la regla — tú la auditas

Velocidad de la IA + verdad del validador + criterio humano



Fallas típicas del borrador: alucina IOCs que el informe no menciona, condiciones flojas (any of) o rígidas (AND que pierde la empacada). El validador manda, no el modelo. CTI sensible → modelo local.

El taller de hoy · tres partes

Caso Locked3D: del informe CTI a las reglas validadas

Partes 1-2 · A mano

- generar_dataset.py → muestras + logs
- YARA: 4 variantes, 0 FP (trampa: runbook)
- Sigma: vssadmin delete vs. backup legítimo
- Ejercicio: la regla de wevtutil cl



Parte 3 · Copiloto + auditoría

- El LLM local redacta desde el informe
- Checklist de auditoría + banco de pruebas
- Iterar con feedback dirigido (2-3 rondas)
- yara y cazar_en_logs.py son los jueces

Próxima sesión · qué viene y qué alistar

Sesión 4 — Bóveda inmutable (WORM)

En la Sesión 4 veremos

- Backups que el ransomware no puede tocar
- MinIO Object Lock (WORM) + restic / Borg
- Inmutabilidad y versionado en la práctica
- Del lab a AWS / Azure (Object Lock real)
- Restaurar con confianza tras la detección

Qué debes traer listo

- Las reglas de hoy funcionando (YARA + Sigma)
- Entorno del kit al día (make check)
- El repo actualizado (git pull)
- Modelo local operativo o API configurada
- Dudas de esta sesión resueltas

¿Preguntas antes del laboratorio?



Idea para llevarse: las firmas de un archivo caducan; las reglas sobre lo que DUELE cambiar (formato, mutex, comportamiento) cazan a la familia. El LLM redacta — el banco de pruebas decide.